

Lexical and Syntax Analysis

Lexical Analysis

Lexical analysis involves reading the source code character from left to right and organizing them into tokens. It aims to read the input code and break it into meaningful elements called *tokens* for a computer to understand easily. It eliminates comments and whitespace within the source code.

A **lexical analyzer** collects characters into logical groupings and assigns internal codes to the groupings based on their structure. These logical groupings are called *lexemes*, and the internal codes for categories of these groupings are the *tokens*.

Here is an example of how the lexical analyzer works.

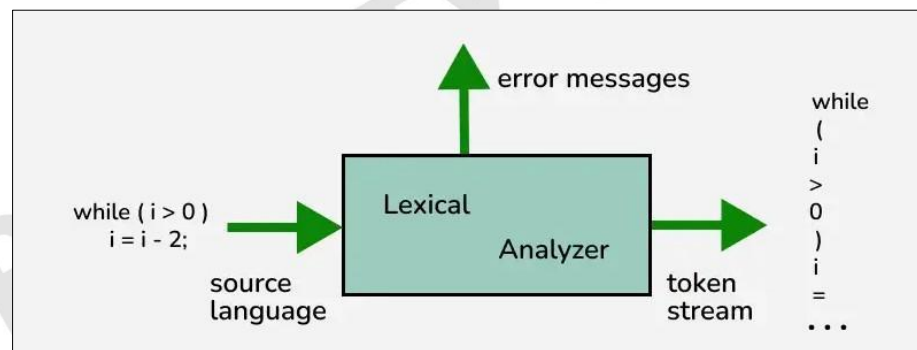


Figure 1. Lexical analyzer. Retrieved from <https://www.geeksforgeeks.org/introduction-of-lexical-analysis/>

In programming language, tokens can be described using **regular expressions**. A lexical analyzer uses a Deterministic Finite Automaton (DFA) to recognize these tokens, as they can identify regular languages. Each final state of the DFA corresponds to a specific token type, allowing the analyzer to classify the input. The process of creating a DFA from regular expressions can be automated to make handling token recognition easier.

Specifically, a lexical analyzer works based on the following processes:

- **Input Preprocessing:** Involves cleaning up the input text and preparing it for lexical analysis. This covers the removal of comments, whitespaces, and other non-essential characters from the input text.
- **Tokenization:** Involves the process of breaking the input text into a sequence of tokens. This is done by matching the characters in the input text against a set of patterns or regular expressions that define the different types of tokens.
- **Token Classification:** The analyzer determines the type of each token. For instance, the analyzer might classify the keywords, identifiers, operators, and punctuation symbols as separate token types.
- **Token Validation:** The analyzer checks if each token is valid based on the rules of the programming language. For instance, the analyzer might check that a variable name is a valid identifier or that an operator has the correct syntax.
- **Output Generation:** The analyzer generates the output of the lexical analysis process, typically a list or sequence of tokens (**token stream**). The list of tokens can then be passed to the next stage of compilation or interpretation, which will be sent to the parser for syntax analysis.

Tokens can be individual words or symbols in a sentence, such as keywords, variable names, numbers, and punctuation. Tokens can be specified in different sets:

- **Alphabets:** All the numbers and alphabets are considered hexadecimal alphabets by language.
- **Strings:** The collection of different alphabets occurring continuously. The string length is defined by the number of characters or alphabets occurring together. For example, the length of |STIisthebest| is 12 since there are 12 characters.
- **Symbols:** High-level programming languages contain special symbols such as the following:

Name	Symbols
Punctuation	Comma (,), Semicolon(:)
Assignment	=
Special Assignment	+=, -=, *=, /=
Comparison	==, !=, <, >, ≤, ≥
Preprocessor	#
Location Specifier	&
Logical	&&, , !
Shift Operator	>>, <<, >>>, <<<

- **Non-tokens:** Comments, preprocessor directive, macros, blanks, tabs, newlines.

Lexemes are the sequence of characters matched by a pattern to form the token or a sequence of input characters that comprises a single token. Additionally, lexemes are recognized by matching the input character string against character string patterns, while tokens are represented as integer values.

Using this assignment statement as an example: `result = oldsum - value / 50;`

These are tokens and lexemes of this statement:

Tokens	Lexemes
IDENT	result
ASSING_OP	=
IDENT	oldsum
SUB_OP	-
IDENT	value
DIV_OP	/
INT_LIT	50
SEMICOLON	;

Using this program as an example:

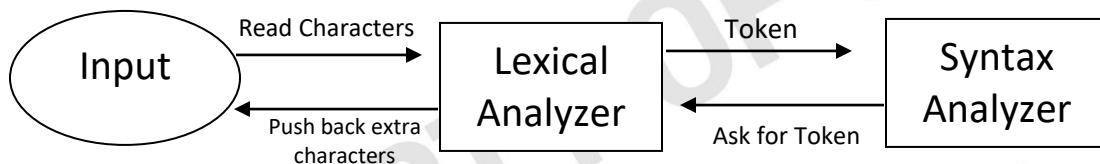
<pre>int main(){ // 2 variables int x, y; x = 10; return 0; }</pre>	There are 18 valid tokens in this program: 'int' 'main' '(' ')' '{' 'int' 'a' ',' 'b' ';' 'a' '=' '10' ';' 'return' '0' ';' '}'. Notice how the comments are omitted.
---	--

Note that everything inside a double quote (") in print() statements is counted as a single token. For example, `println("Walking is a good exercise");` has five (5) tokens: 'println' '(' '"Walking is a good exercise"' ')' and ';'.

The code snippet below has 27 tokens:

```
int main() {
    int x = 15, y = 40;
    printf("sum is:%d", x + y);
    return 0;
}
```

As mentioned, the output (token stream) generated from lexical analysis will be sent to the syntax analyzer for syntax analysis. A zoomed look at how a lexical analyzer and syntax analyzer interact:



Parsing

Syntax analysis, or **parsing**, is the process of analyzing a string of symbols according to the rules of formal grammar. It checks the source code to ensure that it follows the correct syntax of the programming language it is written. **Syntax errors** are identified and flagged in this phase and must be corrected before the program can be successfully compiled.

As mentioned, and as seen in *Figure 2*, it is the phase after the lexical analysis in the compiling process. A **syntax analyzer** or **parser** takes the token streams from a lexical analyzer and analyzes them against production rules to detect errors in the code. A **parse tree** or **Abstract Syntax Tree (AST)** is the output of this phase, representing the program's structure.

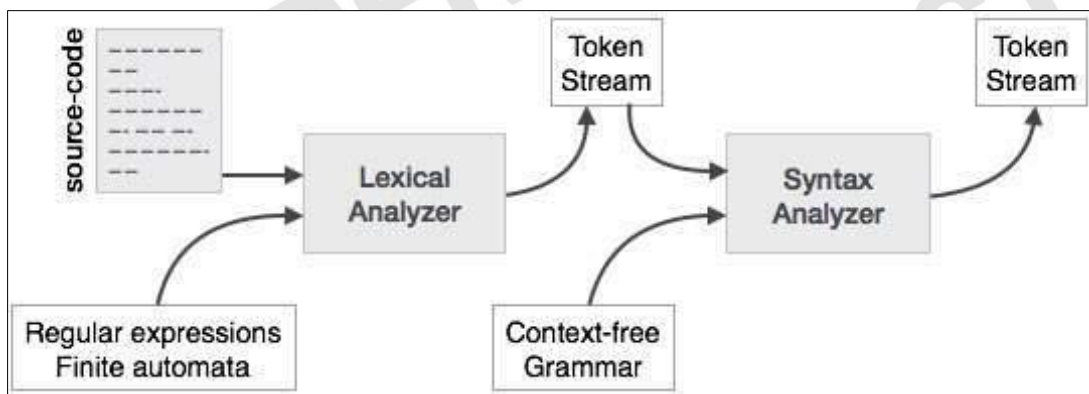


Figure 2. Syntax analyzer. Retrieved from https://www.tutorialspoint.com/compiler_design/compiler_design_syntax_analysis.htm.

A lexical analyzer can identify tokens using regular expressions and pattern rules. Still, it cannot check the syntax of a given sentence since regular expressions cannot check balancing tokens such as parenthesis.

As a result, syntax analysis uses **context-free grammar (CFG)** to define the syntax rules of a programming language. They include production rules that describe how valid strings (token streams) are formed. CFGs also specify the grammar of a language to ensure that the source code adheres to the language's syntax.

The parser accomplishes the following steps:

- **Parsing:** The tokens are analyzed based on the grammar rules of the programming language. A parse tree or AST is constructed to represent the hierarchical structure of the program.
- **Error Handling:** If the input program contains syntax errors, the syntax analyzer detects and flags them to the user, indicating where the error occurred.
- **Symbol Table Creation:** The syntax analyzer creates a symbol table, a data structure that stores information about the identifiers used in the program, such as type, scope, and location.

Derivation

It is the process of applying the rules of Context-Free Grammar to generate a sequence of tokens to form a valid structure. Simply, it is a sequence of production rules to get the input string for the parser.

There are two (2) decisions for some sentential form of input during parsing:

- Deciding on the non-terminal to be replaced
- Deciding the production rule by which the non-terminal will be replaced

There are two (2) options to use in deciding which non-terminal to be replaced with the production rule: Left-most and right-most derivation.

- It is called **left-most derivation** if the sentential form of an input is scanned and replaced from left to right. Its derived sentential form is called the **left-sentential form**.
- It is called the **right-most derivation** if the input is scanned and replaced with production rules. Its derived sentential form is called the **right-sentential form**.

For example, these production rules:

Production Rules	Left-most Derivation	Right-most Derivation
$E \rightarrow E + E$ $E \rightarrow E * E$ $E \rightarrow id$	$E \rightarrow E * E$ $E \rightarrow E + E * E$ $E \rightarrow id + E * E$ $E \rightarrow id + id * E$ $E \rightarrow id + id * id$	$E \rightarrow E + E$ $E \rightarrow E + E * E$ $E \rightarrow E + E * id$ $E \rightarrow E + id * id$ $E \rightarrow id + id * id$
Input string: $id + id * id$	The left-most side non-terminal is always processed first.	The right-most side non-terminal is always processed first.

Parse Tree

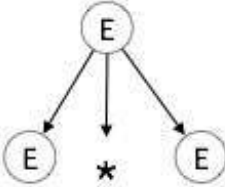
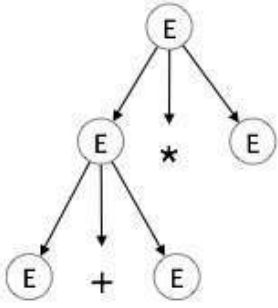
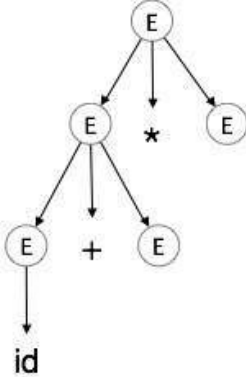
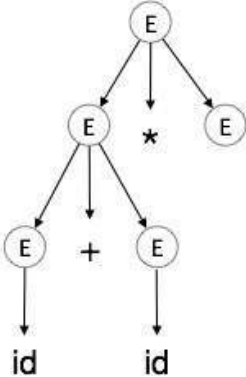
It is the graphical representation of a derivation. It is convenient to see how strings are derived from the start symbol, which becomes the root of the parse tree.

In a parse tree, all leaf nodes are terminals, while all interior nodes are non-terminals. Also, the in-order traversal gives the original input string.

A parse tree represents associativity and precedence of operators. The deepest sub-tree is traversed first, allowing the operator in that sub-tree to get precedence over the operator in the parent nodes.

For example, the left-most derivation of $a + b * c$:

$E \rightarrow E * E$
 $E \rightarrow E + E * E$
 $E \rightarrow id + E * E$
 $E \rightarrow id + id * E$
 $E \rightarrow id + id * id$

Left-most Derivation	Parse Tree
Step 1: $E \rightarrow E * E$	 <pre> graph TD E1((E)) --> E2((E)) E1 --> S1[*] E1 --> E3((E)) </pre>
Step 2: $E \rightarrow E + E * E$	 <pre> graph TD E1((E)) --> E2((E)) E1 --> S1[*] E1 --> E3((E)) E2 --> E4((E)) E2 --> S2[+] E2 --> E5((E)) </pre>
Step 3: $E \rightarrow id + E * E$	 <pre> graph TD E1((E)) --> E2((E)) E1 --> S1[*] E1 --> E3((E)) E2 --> E4((E)) E2 --> S2[+] E2 --> E5((E)) E4 --> ID1[id] </pre>
Step 4: $E \rightarrow id + id * E$	 <pre> graph TD E1((E)) --> E2((E)) E1 --> S1[*] E1 --> E3((E)) E2 --> E4((E)) E2 --> S2[+] E2 --> E5((E)) E4 --> ID1[id] E5 --> ID2[id] </pre>

Left-most Derivation	Parse Tree
Step 5: $E \rightarrow id + id * id$	

Ambiguity

The grammar is ambiguous if it has more than one parse tree, either left or right derivation, for at least one (1) string. For example:

Production Rules	Left-most Derivation	Right-most Derivation
$E \rightarrow E + E$ $E \rightarrow E - E$ $E \rightarrow id$ Input string: $id + id - id$	$E \rightarrow E + E$ $E \rightarrow E - E + E$ $E \rightarrow id - E + E$ $E \rightarrow id - id + E$ $E \rightarrow id - id + id$	$E \rightarrow E - E$ $E \rightarrow E - E + E$ $E \rightarrow E - E + id$ $E \rightarrow E - id + id$ $E \rightarrow id - id + id$
Parse Tree		

The language generated by ambiguous grammar is **inherently ambiguous**. No method can detect and remove ambiguity automatically. Still, it can be removed by either re-writing the whole grammar without ambiguity or by setting and following *associativity* and *precedence* constraints. These methods decrease the chances of ambiguity in a language or its grammar.

Associativity

When an operand has operators on both sides, the side on which the operator takes this operand is decided by the association of those operators. The operand will be taken by the left operator if the operation is *left-associative*, and the right operator will take the operand if the operation is *right-associative*.

Left-associative operations include Addition, Multiplication, Subtraction, and Division.

For example: id op id op id will be evaluated as $(\text{id op id}) \text{ op id}$

Simply, $2 + 3 + 4$ will be evaluated as $(2 + 3) + 4$

Right-associative operations such as exponentiation will have the following evaluation in the same expression as above.

For example: id op id op id will be evaluated as $\text{id op} (\text{id op id})$

Simply, $2 \wedge 3 \wedge 4$ will be evaluated as $2 \wedge (3 \wedge 4)$

Precedence

When two (2) different operators share a common operand, the precedence of operators decides which will take the operand.

For example, $2 + 3 * 4$ can have two (2) different parse trees: one for $(2 + 3) * 4$ and another for $2 + (3 * 4)$. This can be removed by setting precedence among operators. As in the previous example, mathematically, Multiplication (*) has precedence over Addition (+), so the expression $2 + 3 * 4$ will always be interpreted as $2 + (3 * 4)$.

In Python, some operators are performed before others. It is called **the hierarchy of priorities**.

Priority	Operator
1	** or Exponentiation Operator
2	Unary + and –, where unary operators located next to the right of the power operator bind more strongly. For example, $4 ** -1$ equals 0.25.
3	*, /, //, %
4	Binary + (Addition) and – (Subtraction)

This table enumerates the operators in order from the highest (1) to lowest (4) priorities.

References:

- Geeks for Geeks. (2024). *Introduction to syntax analysis in compiler design*. [Web Article]. Retrieved on January 28, 2025, from <https://www.geeksforgeeks.org/introduction-to-syntax-analysis-in-compiler-design/>
- Sebesta, R. (2019). *Concepts of programming languages*. Pearsons
- TutorialsPoint. (2024). *Compiler design – Syntax analysis*. [Web Article]. Retrieved on January 30, 2025, from https://www.tutorialspoint.com/compiler_design/compiler_design_syntax_analysis.htm